

Timeline Progetto Virtual-Staining 2025

Purtroppo ho pensato una settimana troppo tardi di creare questo file per tenere traccia in maniera rigorosa e più dettagliata del lavoro svolto, quindi nel lasso temporale tra il 14/03 e il 19/03 saranno presenti tutte le cose fatte (con problemi riscontrati e soluzioni attuate).

Per riscontri dettagliati vedere i notebook.

Legenda colori:

- **Problemi riscontrati** – criticità rilevate nel processo
 - **Soluzioni attuate** – strategie o interventi correttivi
 - **Futura implementazione/risultati ottenuti** – idee o ottimizzazioni da valutare
 - **Parole chiave** – concetti centrali del progetto
 - **Metodi applicati** – tecniche e algoritmi impiegati
-

11 Marzo

- **Inizio progetto**
- **Assegnazione lettura** paper

Dal 14 Marzo al 19 Marzo

- **Creazione repository**

Problema riscontrato : durante la discussione valutativa iniziale sul progetto è emersa la mancanza di un dataset adibito al training e testing della rete neurale da addestrare per eseguire il virtual staining.

Soluzione proposta: dato che ci sono state fornite alcune immagini di grandi dimensioni, seguendo anche i consigli dei docenti, abbiamo deciso di lavorare con sottosezioni di essa.

Problema riscontrato: le immagini fornite non erano coregistrate e ciò avrebbe indotto degli errori durante la fase di training dell'algoritmo.

Soluzione attuata: sviluppo programma per la coregistrazione delle immagini.

- **Selezione sottosezione comunque tra l'immagine label-free e stained**
- **Valutazione metodi di normalizzazione**

Spiegazione: abbiamo optato, come primo step (in realtà secondo dato che il primo è la conversione in scala di grigi) di effettuare il processo di normalizzazione delle foto. Ciò è dovuto sia a una questione legata alla robustezza del programma: normalizzare tutte le immagini permette un confronto effettivo tra esse, risolvendo alcuni problemi dettati dalla scannerizzazione (diverse tipologie di scanner) e illuminazione. Inoltre, dato che la coregistrazione si basa sullo sviluppo di una matrice omografica (trasformazione geometrica che mappa punti da un'immagine a un'altra quando le immagini rappresentano lo stesso soggetto ma da prospettive differenti), e di conseguenza dell'identificazione e confronto di feature (*feature matching*), abbiamo ritenuto che normalizzare le immagini potesse migliorare la ricerca (dato un incremento di contrasto) degli elementi morfologici più distintivi dell'immagine.

Dai test svolti abbiamo deciso di usare la normalizzazione CLAHE.

- **Valutazione algoritmo di Feature match**

Spiegazione: abbiamo scelto come soluzione per la coregistrazione delle immagini quello di applicare algoritmi di feature match per via della semplicità di implementazione e per via del buon compromesso tempo-risultati. **Una valida opzione da considerare in futuro potrebbe essere quella di applicare algoritmi di tipo Intensity-based.** Abbiamo eseguito test sia su **ORB** che su **SIFT**; entrambi potrebbero essere usati (dipende dalle risorse della macchina e dalla grandezza dell'immagine).

- **Applicazione filtri**

Spiegazione: per iniziare una fase di scrematura abbiamo utilizzato due principali filtri: **Lowe's Ratio** e **Distanza Euclidea**.

- **Filtraggio con RANSAC**

Spiegazione: **RANSAC** è un algoritmo che ci permette di rendere la nostra matrice omografica più robusta attraverso l'eliminazione dei valori anomali (**outlier**) dal nostro insieme di match tra feature.

- **Rifinimento con ECC**

Spiegazione: È un algoritmo di registrazione di immagini basato sull'intensità, che cerca di massimizzare la correlazione tra due immagini per trovare la migliore trasformazione geometrica. A differenza di ORB o SIFT cerca di allineare due immagini trovando la trasformazione che massimizza la somiglianza globale tra i loro pixel, senza usare punti chiave o feature.

I risultati ottenuti finora non lo prevedono; l'abbiamo testato e tenuto in considerazione ma non ancora applicato.

- **Verifica della coregistrazione**

Spiegazione: una volta eseguito il processo sopra indicato abbiamo valutato i risultati ottenuti attraverso dei criteri standard come **Mutual Information** e **Mean Square Error**. Inoltre abbiamo ritenuto valido il confronto degli istogrammi dell'immagine label-free in scala di grigi con la differenza assoluta ottenuta tra label-free e stained allineata. Questo processo confronta il numero di pixel neri presenti nell'immagine originale con quella allineata (un pixel perfettamente allineato assume il valore 0). È evidentemente un metodo immediato per osservare il miglioramento.

- **Ritaglio finale**

Problema riscontrato: applicando il programma sviluppato a una serie di immagini abbiamo osservato come molte di esse venissero allineate in maniera sufficientemente valida, ma alcune no. Questa discontinuità nei risultati è dovuta alla presenza dello sfondo e da come esso venga mal gestito dalla normalizzazione, causando una (presunta) creazione di false feature e di conseguenza non permettendo agli algoritmi di funzionare correttamente.

Possibile soluzione: due possibili soluzioni possono essere o il riconoscimento del bordo della cellula e un conseguente isolamento dello sfondo, sul quale non applicheremmo la normalizzazione, oppure l'applicazione di algoritmi differenti come ECC oppure Intensity-based.

Questo problema e questa soluzione ancora non sono stati affrontati. Saranno affrontati probabilmente nella settimana del 24 Marzo.

24 Marzo

Problema riscontrato: eseguendo diversi test su sottoimmagini abbiamo notato come si creassero degli artefatti dovuti alla normalizzazione in zone di sfondo o genericamente bianche e grandi (rispetto alla dimensione delle immagini). Questo induceva l'algoritmo **SIFT** (o ORB) a cercare delle feature in queste zone,

rischiando di conseguenza di trovare feature inesistenti.

Soluzione proposta: abbiamo optato, seguendo il consiglio dei docenti, per lo sviluppo di una maschera che permettesse di evitare di normalizzare eventuali zone critiche e di ricercare in esse feature. In sintesi abbiamo applicato la maschera sia alla normalizzazione che a SIFT.

- **Abbiamo valutato diversi metodi per creare la maschera**

Spiegazione: abbiamo valutato principalmente tre metodi: il **Flood Fill from Edges**, **Connected Components** e infine un possibile **White pixel counter**.

Problema riscontrato: il white pixel counter, per via della sua eccessiva semplicità non otteneva i risultati desiderati, e di conseguenza l'abbiamo scartato come opzione; il flood fill from edges poteva essere una valida opzione ma, in corso d'opera, abbiamo deciso di considerare all'interno della maschera anche le zone bianche interne all'immagine.

Soluzione attuata: utilizzo di connected components per l'ottenimento della maschera finale.

- **Binarizzazione dell'immagine per rilevamento dei componenti**

Spiegazione: binarizzando l'immagine imponendo una soglia di 230 (valore ottenuto sperimentalmente e osservando i valori di intensità all'interno dell'immagine) abbiamo ottenuto una matrice binaria che descrive una maschera primitiva da filtrare successivamente (per via dei troppi dettagli trascurabili)

- **Inizio dell'analisi dei componenti**

Spiegazione: una volta ottenuta la matrice binaria abbiamo filtrato le zone più piccole e ininfluenti considerando come criterio il fatto che lo sfondo debba essere abbastanza grande e omogeneo; in questo modo filtriamo tutte le zone più piccole presenti o non omogenee.

Problema riscontrato: come capiamo se le zone sono omogenee?

Soluzione attuata: attraverso un soglia di deviazione standard possiamo filtrare gruppi candidati alla partecipazione della maschera. I gruppi che presentano rumore o poco omogenei saranno scartati; questo ci permette inoltre di mantenere zone interne della cellula contenenti informazioni.

25 Marzo

- **Creazione script per ottenere la maschera dell'immagine originale (20k x 20k)**

Spiegazione: applicare l'algoritmo per la creazione della maschera sull'immagine originale non era possibile per via delle risorse hardware insufficienti, di conseguenza abbiamo diviso l'immagine in n sottoimmagini e applicato ad esse l'algoritmo per la maschera. Dividendo l'immagine originale in quarti (ad esempio) abbiamo ricavato nella prima riga tre quadrati ($[0, 0.5 \times \text{Tot}]$, $[0.25 \times \text{Tot}, 0.75 \times \text{Tot}]$, $[0.5 \times \text{Tot}, \text{Tot}]$), in questo modo abbiamo ottenuto, sovrapponendo attraverso un AND logico le rispettive maschere, una robustezza maggiore e una sovrapposizione corretta.

26 Marzo

- **Applicazione maschera in fase di allineamento**

Spiegazione: dopo aver rifinito l'algoritmo per la creazione della maschera e testato ulteriori parametri (*threshold* e *stddev_threshold*) l'abbiamo applicato alle sottoimmagini dell'immagine originale e osservato come in realtà **il miglioramento ottenuto tramite l'ausilio della maschera è veramente minimo**.

- **Creazione griglia senza sovrapposizione**

Spiegazione: abbiamo creato un breve script per ottenere un numero n di coppie di sottoimmagini label-free/stained. **La griglia applicata al momento è abbastanza elementare e non presenta elementi di ridondanza (al contrario della griglia per la maschera fatta il 25 Marzo) nel posizionamento dei quadrati. In futuro sarebbe meglio introdurre ridondanza per aumentare la robustezza.**

Problema riscontrato: volevamo costruire il nostro dataset costituito dalle sottoimmagini dell'immagine originale, ma applicando lo script sopra citato abbiamo notato come le coppie di immagini non fossero (ovviamente) coregistrate, ciò è dovuto al fatto che il processo di coregistrazione sarebbe stato applicato dopo.

Soluzione proposta: possiamo applicare una coregistrazione meno raffinata giusto per avere un miglior punto di partenza.

Problema riscontrato: ECC sull'immagine 20k x 20k non ha effetto.

Soluzione proposta: applicare ECC su quarti (o sedicesimi) dell'immagine.

Problema riscontrato: ECC non ha effetto nemmeno su sottosezioni dell'immagine.

Soluzione attuata: Non facciamo l'allineamento iniziale e ci limitiamo a eseguire la coregistrazione su ogni sottoimmagine.

- **Creazione script suddivisioni immagine e script coregistrazione**

Spiegazione: lo script `grid_script.py` genera una cartella contenente n coppie di immagini label-free/stained e n coppie di maschere corrispondenti. Lo script `alignment_script.py` prende le immagini dalla cartella `grid/` (creata dallo script precedente), le allinea e le colloca nella cartella `aligned/`. Le coppie che non soddisfano i requisiti (almeno il 60% di immagine e non sfondo, e abbastanza match) vengono collocate in `bad_alignment/`.

28 Marzo

- **Creazione script convalida dataset**

Spiegazione: lo script verifica che, per ogni immagine, sia presente la sua corrispettiva stained, verifica che le dimensioni delle immagini costituenti le coppie siano uguali. Infine plotta dei grafici rappresentanti l'andamento delle differenze assolute del dataset.

31 Marzo

- **Colloquio con i docenti per discutere sulle prossime cose da sviluppare o da rivedere**

Esito colloquio: dal colloquio è emerso un esito positivo da parte dei docenti, i quali ci hanno proposto di continuare lo sviluppo del progetto, iniziando a pensare alla realizzazione della rete pensando a una modellizzazione **GAN** o a un **Diffusion model**. Durante il colloquio è emersa anche un'idea sull'allineamento svolto e che ci avrebbe permesso di allineare le immagini 20k x 20k.

- **Sviluppo notebook (diventerà uno script) per la suddivisione in cartelle per la rete**

Spiegazione: All'interno del notebook è riportato un breve script che crea tre cartelle `train/`, `val/` e `test/` contenenti le coppie di immagini adibite rispettivamente al **training**, **validation** e **testing**. La cartelle conterranno il 70%-15%-15% delle coppie originali. Le immagini inserite all'interno delle cartelle non sono contigue, ma bensì prese casualmente.

- **Sviluppo idea per la coregistrazione delle immagini intere**

Problema riscontrato: la coregistrazione delle singole sottoimmagini non ci avrebbe permesso di ricostruire l'immagine originale allineata, per via delle trasformazioni applicate. **Soluzione attuata:** abbiamo allineato l'immagine originale applicando la matrice omografica, calcolata sull'immagine originale scalata (metà risoluzione), sull'immagine stessa.

- **Impostazione dell'ambiente di sviluppo per PyTorch**

Spiegazione: durante il pomeriggio della giornata abbiamo provato a lavorare parallelamente alle due voci dell'elenco puntato (idea per la coregistrazione e settaggio per PyTorch). Abbiamo verificato come scaricare **PyTorch** e come impostarlo affinché potesse girare su GPU (Nvidia RTX 3060Ti) sfruttando i **Cuda Cores**. Abbiamo provato precedentemente a far girare anche alcune istruzioni di **OpenCV** su scheda grafica, ma senza successo; **bisognerebbe compilare manualmente il pacchetto per poter sfruttare questa funzionalità; si potrebbe fare in futuro per velocizzare il processo iniziale.**

1 Aprile

- **Creazione script di prova per vedere il funzionamento generale di PyTorch**

Spiegazione: presi dalla curiosità ci siamo messi a verificare il comportamento di una semplice rete di convoluzione (collocata all'interno dello script `prova_pytorch.py`). Volevamo capire la struttura fondamentale di un algoritmo scritto in PyTorch e se il tentativo svolto il giorno prima (per farlo girare su GPU) funzionasse.

- **Creazione script di prova per vedere il funzionamento di pix2pix**

Spiegazione: dato il "successo" di esecuzione dello script precedente, abbiamo provato a creare un semplice script basato sul modello **GAN**, quindi con un **Generator** e un **Discriminator**, creati con reti differenti (**UNet-like** e **PatchGAN**). I risultati ottenuti in fase di training sembrano positivi e sulla giusta strada, ma ancora ben distanti dai risultati che vogliamo ottenere (dato che i colori non sono sempre assegnati correttamente). **I prossimi passi dovrebbero essere quelli che ci permettono di migliorare questo prototipo (inserendo più layer o modelli più approfonditi per Generator e Discriminator).**

2 Aprile

- **Ricerca sul funzionamento algoritmi**

- **UNet e PatchGAN aggiornati**

Spiegazione: Le reti del generatore e del discriminatore sono state aggiornate per essere più profonde e per permetterci di ottenere risultati più accurati. Abbiamo aumentato il numero dei **layer** della rete, arrivando a 4, e aggiungendo **skip connections**. Anche il discriminatore ha ricevuto più layer.

- **Ottimizzazione rete con Automatic Mixed Precision (AMP)**

Problema riscontrato: l'allenamento della rete era troppo lento per poter procedere a passo sostenuto con lo sviluppo. **Soluzione attuata:** abbiamo eseguito un processo di ottimizzazione della rete implementando **AMP**, il quale sfrutta i **Tensor Cores** della scheda video NVidia, applicando a determinate operazioni la precisione a 16 bit invece che a 32. I risultati ottenuti sono stati soddisfacenti e i tempi sono notevolmente diminuiti.

3 Aprile

- **Aggiunta commenti sullo script**

- **Aggiunti i checkpoint della rete**

Spiegazione: ogni n volte (`checkpoint_rate`) la rete crea un checkpoint (implementato tramite un dizionario) il quale ci permette di salvare lo stato attuale della rete e di ricaricarlo per, o riprendere l'allenamento, o eseguire la fase di Test.

- **Aggiunta funzione di validation**

Spiegazione: ogni m volte (`validation_rate`) la rete crea e esegue un test di validazione su delle apposite immagini contenute nella cartella `dataset_split/val/`. Questa operazione ci permette di monitorare l'andamento della fase di training della rete.

- **Parametrizzazione codice**

Spiegazione: per comodità abbiamo inserito dei parametri a inizio codice che sostituiscono i numeri inseriti nel codice. In questo modo possiamo modificare i parametri più facilmente e velocemente ed eseguire diversi test in diverse condizioni.

- **Diverse prove di training**

4 Aprile

- **Aggiunta documentazione sull'avvio sequenziale degli script**

Spiegazione: data la complessità degli script, e dato che stavamo iniziando a confonderci, abbiamo deciso di scrivere sul file `README.md` una sezione apposita per descrivere il processo di creazione. **In futuro dovremo creare un singolo file eseguibile che fa tutto.**

- **Creazione script per il confronto delle immagini di test generate**

Spiegazione: è stato creato uno script base per creare immagini formate da `[input|output|target]`. Le prime sono le immagini *label_free*, le seconde indicano i risultati generati dalla rete, mentre gli ultimi sono le immagini *stained* reali.

- **Inizio sviluppo UNet 3+ e Advanced PatchGAN**

Spiegazione: dopo svariate prove abbiamo raggiunto quello che riteniamo sia il limite massimo di apprendimento della rete neurale composta da UNet e PatchGAN. Di conseguenza abbiamo provato a sviluppare una seconda rete neurale sfruttando i modelli UNet 3+ e Advanced PatchGAN. Il file tentativo che contiene questa rete è `Pix2Pix++.py`. Il nome della rete è dovuto all'ispirazione dal C e dal C++.

5 Aprile

- **Test Pix2Pix++**

Spiegazione: abbiamo testato la rete per verificare i risultati ottenuti e non hanno minimamente rispettato le aspettative. **Probabilmente i risultati scarsi sono dovuti a una metrica implementata male. In futuro dovremo o sistemarla o rimuoverla.**

6 Aprile

- **Rimozione metrica WGAN-GP da Pix2Pix+**

Spiegazione: abbiamo rimosso la metrica che creava problemi.

7 Aprile

- **Creazione script `ollie_wan_kenobi.py`**

Lore: data la presenza di troppi file abbiamo deciso di creare un unico file contenente tutte le precedenti operazioni (allineamento, creazione dataset...). Non sapendo che nome dargli abbiamo optato per un'ispirazione al mondo tech, più precisamente ai computer, infatti il primo nome proposto è stato `all_in_one.py`. Dato che siamo dei burloni abbiamo trasposto il nome in `ollie_wan.py`. Il nome però faceva chiaramente pensare a un personaggio iconico del mondo cinematografico e ludico, di conseguenza il nome finale proposto per lo script che sarà eseguito in preparazione alla rete è `ollie_wan_kenobi.py`. Nessuno dei due creatori di questo file e repository è fan, e tanto meno ha visto, Star Wars.

8-9-10 Aprile

- **Refactoring codice**

14 Maggio

- **Riorganizzazione repository**

Spiegazione: dopo un periodo di interruzione dovuto a impegni non rimandabili di natura universitaria e personale, abbiamo ripreso in mano il progetto con l'idea di riorganizzare la repository, "pulendola" da file inutili e tentativi momentanei di idee mai sviluppate. **Essa presenterà 2 script** (`ollie_wan_kenobi.py` e `Pix2Pix.py`) **e 3 notebook di spiegazione approfondita su essi** (`allineamento.ipynb`, `ollie_wan_kenobi.ipynb` e `Pix2Pix.ipynb`). Tale giorno abbiamo finito la stesura del primo notebook nominato e scritto la relazione finale del progetto ai fini di terminarlo formalmente.